next-workflow 2.5

A step-by-step guide to adding this Claude Code workflow to your Next.js app, and what it does once it's in.

---

1. What is next-workflow?
2. What you need first
3. Installing it: plugin or project files
4. Your first task: plan, build
5. How a task is checked
6. Running many tasks safely
7. Projects with their own stack
8. All commands and helpers
9. The specialist agents
10. Automatic checks (hooks)
11. The coding rules it follows
12. Settings you can change
13. A typical day
14. Problems and fixes
15. What's new in 2.5
16. Measuring the workflow

1. What is next-workflow?

next-workflow is a **kit for Claude Code**, not a Next.js library. Nothing is added to your app's code or bundle. It adds instructions and checks to Claude Code, as a plugin or as files in your project's `.claude/` folder, and they change how Claude Code works on your Next.js project.

- **It plans before it builds.** Opus turns your request into small task files with clear "done" checks. Read them and change anything you disagree with.
- **Sonnet builds each task.** It works directly for small tasks and uses specialist agents for large ones.
- **A second agent checks the work.** It sees only the task and the changes, never the builder's reasoning. A task can't be closed without its verdict.
- **It fits your project.** It detects your Next.js, Tailwind and library versions. In a Redux, antd or formik project it follows those, not its own defaults.
- **Automatic checks run on every edit** and catch Next.js mistakes before the build does.
- **It remembers your project** in `.claude/project-context.md`, and recent verification failures, so it doesn't re-learn or repeat them.
- **It's safe to undo.** You can get one commit per task and roll back any single task.

> **In one sentence**
>
> Describe what you want with `/next:write-task`, run `/next:task`, and every task is built, independently checked and recorded.

2. What you need first

Requirement	Why
A Next.js 15 or 16 project	Its `package.json` must list `"next"`. The workflow detects the version and follows the right rules.
Claude Code	The workflow runs inside it (terminal, desktop app or IDE).
bash, Node.js	The checks are shell scripts (Linux, macOS, or WSL on Windows).
`jq`	Needed for safe settings merging, the track record and batch runs.
git (recommended)	Enables task diffs, the verifier's view of changes, per-task commits and undo.
Playwright (optional)	Lets the verifier check `[browser]` criteria in a real browser.

3. Installing it: plugin or project files

There are two ways. Pick **one** per project. If both are present, the plugin says so at session start and the project files win.

	A. Plugin new in 2.5	B. Project files
Where it lives	Once, in Claude Code. Nothing is copied into the project.	Copied into `your-app/.claude/`, so it can be committed for the team.
Updating	`claude plugin update next@next-workflow`	Run `./install.sh` again.
Helper commands	In the plugin's folder. The session report prints the path.	`bash .claude/hooks/…`

A Install as a plugin

```
claude plugin marketplace add ~/www/hobbies/next-workflow
claude plugin install next@next-workflow
```

The commands are the same `/next:*`. A plugin can't carry permission rules, so two extra checks take their place: one approves only the kit's own helper and test commands, and one blocks Claude from reading `.env` files and keys. Your own deny and ask rules always win.

B Or copy it into your app

```
cd ~/www/hobbies/next-workflow
./install.sh /path/to/your-next-app           # add --dry-run to preview
```

This copies the kit into `your-next-app/.claude/` and merges `settings.json`. The old file is backed up and older kit entries are replaced, so nothing is duplicated. It also stamps the kit version and records the type errors that already exist, so Claude is held only to the errors it introduces.

1 Open Claude Code in your app, then run setup once

```
cd /path/to/your-next-app
claude
/next:setup
```

At start you'll see a short report: kit version, Next version, Tailwind, your libraries, test tools and the verify commands. That means it's working. Setup creates `.claude/tasks/`, fills `.claude/project-context.md` with facts about your repo, adds a short pointer to `CLAUDE.md`, and tells you what won't work here (for example "no git, so no undo").

> **If setup says the typecheck is already failing**
>
> Type `! bash .claude/hooks/verify.sh --baseline` in the prompt (plugin: the helpers path from the report). That records the existing errors. Claude isn't allowed to do this itself, so it can never excuse its own mistakes.

> **Commit**
>
> With project files: `.claude/` (including `.claude/tasks/`) and `CLAUDE.md`, so your team gets the same workflow. With the plugin: `.claude/tasks/`, `.claude/project-context.md` and `CLAUDE.md`. `.claude/.cache/` is git-ignored.

4. Your first task: plan, build

1 Describe what you want

```
/next:write-task a projects page where users see their projects and can archive one
```

The planner (Opus) asks questions first if something is unclear, splits big requests into small tasks, sets the order (`depends`) and size, and writes the task files, ready to run. It never writes code.

2 Read the tasks, change what you disagree with

```
---
id: 003
title: Projects page with archive
status: todo
depends: 002
size: large
---
## Goal
Signed-in users see their own projects at /projects and can archive one.

## Acceptance
- [ ] [review]  /projects lists only the signed-in user's projects
- [ ] [test]    Archiving someone else's project fails and changes nothing
- [ ] [browser] Signed-out users are redirected to /login

## Notes
Out of scope: un-archive.
```

Each "done" check carries a tag saying how it will be proven:

- `[test]`: a test must prove it.
- `[browser]`: it has to be seen in the running app.
- `[review]`: reading the code is enough.

3 Build it

```
/next:task
```

Sonnet takes the next runnable task and works through it: builds it (directly if small, with the full phased build if large), runs the checks, gets the verifier's verdict (section 5), closes it with a log line, and saves what it learned.

todo → **doing** → **done** or **blocked**

5. How a task is checked

The builder can't mark its own work done. `tasks.sh close` refuses until a verdict is recorded:

Verdict	What happens
PASS	Every criterion was proven. The task closes.
FAIL	The builder fixes exactly what failed and the verifier checks again. After two failed rounds the task is **blocked**. What failed is recorded, and later sessions see it.

UNVERIFIED	Everything else passed, but a `[browser]` criterion couldn't be seen (no dev server or Playwright). The task closes as done, and its log line names the criteria **you** should check.
SKIPPED	Only for tiny changes: every criterion is `[review]` and the diff is at most 20 lines (you can raise this to 60).

Track record new in 2.5: each session report shows the verifier's first-pass rate and the last three failures ("004: search was case-sensitive"), so Claude doesn't repeat them.

6. Running many tasks safely

Running tasks one after another in a single conversation lets context pile up, and quality drops by the 4th or 5th task. So `/next:task all` stops after 3. For longer batches use the runner, which starts a **fresh session for every task**:

```
bash .claude/hooks/run-tasks.sh --checkpoint          # all runnable tasks, in order
bash .claude/hooks/run-tasks.sh 004 005 --budget 3    # just these, max $3 each
```

Option	Effect
`--checkpoint`	One local git commit per finished task, named `task 004: …`, on a new branch `nw/batch-<time>` so `main` is never touched. Each commit holds only that task's files: never secret files, never edits that were already there.
`--no-branch`	With `--checkpoint`, commit on the current branch instead
`--budget USD`	Spending limit per task
`--max N`	Stop after N tasks
`--dry-run`	Show what would run, and run nothing

The runner stops at the first task that doesn't finish (blocked, failed or over budget). Run it again and it continues from there. It prints time, turns and cost per task, and points out tasks that closed with UNVERIFIED criteria for you to check.

Undo one task (needs `--checkpoint`): `bash .claude/hooks/tasks.sh undo 004`. That task's code is reverted and it goes back to `todo`. If later finished tasks depend on it, undo refuses and lists them. `undo 004 --cascade` undoes those first, newest first.

When the batch looks good: `git switch main && git merge nw/batch-…` (the runner prints the exact command). Claude pushes or opens a pull request only when you ask it to.

7. Projects with their own stack new in 2.5

The kit's own line is server-first, zod and zustand (section 11). Many real apps are built differently. When the project uses any of these, the kit switches to **match mode**:

Kind	Detected
State	Redux Toolkit, redux, jotai (zustand stays on the kit line)
Server data	RTK Query (found from `createApi`), react-query, SWR, Apollo
Forms, validation	formik, TanStack Form, yup, valibot (yup without zod means match)
UI kits	antd, MUI, Chakra, Mantine

In match mode the session report says **"match the existing stack"** and lists your libraries. Claude then:

- follows the patterns already in your code, and never adds a second library for the same job (no zustand next to Redux, no zod next to yup);
- adds data access through your data client (an RTK Query endpoint, a `useQuery` hook), not a fetch in an effect;
- validates Server Actions with the **same** yup schema the form uses;
- doesn't move existing client code to the server unless the task asks;
- still applies the safety rules: authenticate, authorize and validate at every boundary, accessible markup, no secrets in client code.

The rules are in `react-component-conventions/references/existing-stacks.md`. To force a mode, set `NEXT_WF_STACK_MODE=kit` or `match`.

8. All commands and helpers

Slash command	Model	Use it to…
`/next:setup`	current	Connect the workflow to the project. Run once.
`/next:write-task <what>`	Opus	Plan work into task files. Never writes code.
`/next:task [id|all|list]`	Sonnet	Build tasks: your main command.
`/next:feature <id>`	Sonnet	Force the full phased build (data → page → UI → tests → review) on a task.
`/next:test [target]`	current	Write or fix tests outside a task.
`/next:review [target]`	Opus reviewer	Review your changes for bugs and security holes. It doesn't change anything.
`/next:audit [path]`	current	Performance, accessibility and correctness audit, ranked.

`/next:ship`	current	Typecheck, lint, tests and build, fixing what fails. Never commits.

Terminal helpers are below with the project-files path. With the plugin, use the **Kit helpers** path from the session report instead of `.claude/hooks`.

Terminal helper	Does
`bash .claude/hooks/tasks.sh list`	Status of every task
`… tasks.sh diff <id>`	Everything a task changed
`… tasks.sh undo <id> [--cascade]`	Revert a task's checkpoint commit (and its dependents with `--cascade`)
`… tasks.sh map`	A short map of the app (routes, components, lib modules), cached
`… tasks.sh context <id>`	The task plus what its dependencies did and changed
`… tasks.sh stats [--brief]`	Tasks done, time per size, batch cost, verifier first-pass rate, which checks fire most. `--brief`: the one-line track record. 2.5
`… tasks.sh compact-context`	Tidy `project-context.md`: drop duplicate facts, merge repeated headings, remove empty sections (a backup is kept). 2.5
`… tasks.sh dir` / `migrate`	Which folder holds the tasks / move an older project's `tasks/` to `.claude/tasks/`
`bash .claude/hooks/run-tasks.sh`	Batch runner, with a fresh session per task

9. The specialist agents

Agent	Job	Can edit?
`next-route-builder`	Pages, layouts, loading, error and not-found files, metadata	Yes
`next-ui-builder`	Components, styling, accessible markup	Yes
`next-data-layer`	Server Actions, queries, caching, API routes	Yes
`next-test-writer`	Vitest and Playwright tests	Yes
`next-verifier`	Checks every "done" criterion against the task and diff only (Sonnet)	No
`next-reviewer`	Security and correctness review (always Opus)	No
`next-perf-auditor`	Bundle size, waterfalls, images and fonts	No
`next-a11y-auditor`	Keyboard, focus, labels, contrast (no shell access at all)	No

Every agent starts with the stack report and project memory, so it doesn't explore your code from scratch. With the plugin, agents are named `next:next-verifier` and so on.

10. Automatic checks (hooks)

Check	When	What it does
`session-context`	Session start	Reports the kit version and your stack, points to the Next 15 or 16 rules, says whether to match your stack, and loads the project memory and track record.
`post-edit`	After each edit	Runs every edit check: App Router mistakes, component rules, Tailwind classes, then formatting. It gives one combined answer.
`bash-guard`	After shell commands	Files written through the shell (not Claude's edit tool) get the same checks, and still trigger the typecheck.
`guard-secrets`	Before shell commands	Blocks commands that read `.env` files or private keys. `.env.example` is allowed. It slows accidents down but isn't a security boundary (section 14).
`guard-owner`	Before shell commands	Keeps your decisions yours: task statuses change only through `tasks.sh`. Claude can't record the type-error baseline, and can't set the verification switches in section 12.
`verify`	Before Claude finishes	Typecheck. Only **new** errors fail; errors that already existed are baselined.
`guard-read`, `allow-helpers`	Plugin only	Stand in for the permission rules a plugin can't carry: block reading secret files, pre-approve the kit's own helper and test commands. Anything chained or unusual still asks you.

Every check hit is recorded in `.claude/.cache/metrics.jsonl`. `tasks.sh stats` summarises them.

11. The coding rules it follows

- **Server Components by default.** Interactivity goes in small separate "island" files (`*.client.tsx`).
- **Components only render.** Logic goes into hooks, services, helpers and schemas.
- **zod** at every input boundary. **zustand** only when needed, created per request. (In match mode, your own libraries take their place: section 7.)
- **Server Actions are public endpoints:** authenticate, authorize, validate, then change data.
- **Version-aware:** Tailwind classes for the version you actually have installed. On Next 16 it uses `proxy.ts`, `'use cache'` / `cacheLife` / `cacheTag` and `updateTag`, and it greps the docs bundled with Next 16.2+ before relying on memory.
- **Accessibility is part of the job**, and your existing style always wins.

12. Settings you can change

Set these in your shell before starting `claude` (or before the batch runner), or in the `env` section of your Claude Code settings. Claude is blocked from setting the verification switches itself.

Variable	Values	Effect
`NEXT_WF_CHECKPOINT`	`commit`	One local commit per finished task (the runner's `--checkpoint` sets this)
`NEXT_WF_DEV_URL`	e.g. `http://localhost:3000`	Lets the verifier check `[browser]` criteria in a real browser (needs `@playwright/test` in the project; the kit brings its own Playwright config)
`NEXT_WF_VERIFY_BROWSER` 2.5	`on`, `off`	`off` saves the most verifier tokens: no Playwright, and `[browser]` criteria are recorded UNVERIFIED for you to check
`NEXT_WF_SKIP_VERIFY_LINES` 2.5	number, default `20`, max `60`	How small a `[review]`-only change must be to skip the verifier
`NEXT_WF_STACK_MODE` 2.5	`kit`, `match`	Override the detected mode (section 7)
`NEXT_WF_VERIFY`	`typecheck`, `typecheck,lint`, `off`	What runs before Claude finishes
`NEXT_WF_REQUIRE_VERDICT`	`on`, `off`	The verdict gate on `close` (section 5)
`NEXT_WF_COMPONENT_GUARD`	`enforce`, `warn`, `off`	Component rules: block, only advise, or off
`NEXT_WF_STYLE_GUARD`	`on`, `off`	Tailwind check
`NEXT_WF_SECRETS_GUARD`	`on`, `off`	Blocking of `.env`/key reads
`NEXT_WF_FORMAT`	`all`, `prettier`	`prettier` skips the per-edit ESLint fix (faster on big repos)
`NEXT_WF_METRICS`	`on`, `off`	Recording to `metrics.jsonl`

Update: plugin: `claude plugin update next@next-workflow`. Project files: run `./install.sh /path/to/app` again. **Remove:** `claude plugin uninstall next@next-workflow`, or `./install.sh /path/to/app --uninstall`, which removes the kit's files and hook entries and keeps your tasks, memory and own settings.

13. A typical day

1. `/next:write-task …`, then answer its questions.
2. Read the new files in `.claude/tasks/` and change anything you disagree with.

3. Optional: start `npm run dev` and set `NEXT_WF_DEV_URL` so browser checks really run (or `NEXT_WF_VERIFY_BROWSER=off` to save tokens and check them yourself).
4. `bash .claude/hooks/run-tasks.sh --checkpoint` and let it work, or `/next:task` for one at a time.
5. Check the result, including any UNVERIFIED criteria the runner lists. Undo any task you don't like with `tasks.sh undo <id>`.
6. `/next:review`, then `/next:ship`. When everything is green, push, or ask Claude to push or open the pull request.
7. Now and then, run `tasks.sh stats` to see cost, time, the verifier's pass rate and which checks fire most.

14. Problems and fixes

Problem	Fix
Every turn fails on old type errors	Record them once: `! bash .claude/hooks/verify.sh --baseline`
A task is `blocked`	Read the reason in its Log section (in headless runs it's the question Claude would have asked), add the answer to Notes, set it back to `todo`, and run it again.
"no verdict recorded" when closing	Intended: every task needs the verifier's verdict (section 5). Let `/next:task` finish the verification step.
The verifier marks `[browser]` items UNVERIFIED	Start the dev server and set `NEXT_WF_DEV_URL`, with Playwright installed, and make sure `NEXT_WF_VERIFY_BROWSER` isn't `off`.
Claude added zustand/zod to a Redux/yup app	Check the session report says "match the existing stack". If not, set `NEXT_WF_STACK_MODE=match`.
"installed twice" at session start	The project has both the plugin and project files. Keep one: `./install.sh /path/to/app --uninstall`, or disable the plugin.
Plugin: a helper command still asks permission	Only plain helper and test commands are pre-approved. Anything chained, with flags, or matching one of your own ask rules asks on purpose.
Claude was blocked setting a `NEXT_WF_…` variable	Intended: the verification switches are yours. Set them yourself (section 12).
`undo` says there's no checkpoint	The task was closed without `--checkpoint`. Use `tasks.sh diff` and revert by hand.
"checkpoint: SKIPPED — secret files changed"	A `.env` or key file changed during the task. Add it to `.gitignore`, then commit the task's files yourself. Nothing was committed.

"project-context.md … only the first 4000 were loaded"	Run `tasks.sh compact-context`, then merge what still repeats until it's under ~60 lines.
Claude asks permission to edit a file in `.claude/`	Normal: Claude Code protects that folder. New tasks don't ask, because they're written by `tasks.sh new`. Approve edits you asked for, or pick "allow for this session".
A command was blocked for touching `.env`	That's intended. Run it yourself, or use `.env.example`. The guard can't catch everything: for real protection keep production secrets out of the project folder, or use Claude Code's sandbox (`/sandbox`).
A check blocks something you want	Set that check to `warn` or `off` (section 12).
Installer: no "next" dependency	Point it at the folder that holds your app's `package.json`.

15. What's new in 2.5

Change	What it means for you
Match the existing stack	Redux, RTK Query, react-query, formik, yup, antd and similar projects are followed, not rewritten to the kit's defaults (section 7).
Plugin install	Install once, update with one command, nothing copied into projects (section 3).
Cheaper verification	`NEXT_WF_VERIFY_BROWSER=off` and `NEXT_WF_SKIP_VERIFY_LINES`. Verification is never skipped; unchecked browser items come to you as UNVERIFIED.
Track record	Recent verifier failures and the first-pass rate are in every session report.
`compact-context`	Keeps the project memory short, and so cheap to load.
Next 16 baseline	A known-good Next 16 + cacheComponents solution scores 5/5 on a real 16.4.1 build. Found on the way: a `redirect()` inside streamed `<Suspense>` returns HTTP 200, not 307, so auth redirects belong in `proxy.ts`.
Fixes	The stack cache now really works (detection used to re-run on every edit). `pnpm run`, `yarn` and `bun run` no longer ask permission, and `bun.lock` is detected. The batch runner no longer suggests a removed command.

Earlier releases

Version	Main change
2.4	Verdict gate: a task can't close without the verifier's verdict. Tasks run straight from planning (no approve step). Statuses move only through `tasks.sh`.

2.3	Tasks live in `.claude/tasks/`, and are written by `tasks.sh new` with no permission prompt per task.
2.2	Context memory: repo map, task handoff, subagents start with the stack report, memory reloaded after compaction.
2.1	Safe checkpoints (never secrets), real Playwright browser checks, undo with dependents, the eval harness, CHANGELOG and CI.
2.0	The verifier agent, a fresh session per task, per-task commits and undo, the type-error baseline, and metrics.

Every change is listed in `CHANGELOG.md` in the kit.

16. Measuring the workflow

```
bash evals/run.sh --reference             # free: proves every check can pass (must be 5/5)
bash evals/run.sh --reference --next 16   # the same on Next 16 + cacheComponents
bash evals/run.sh                         # 3 real agent runs on Next 15 (costs API money)
bash evals/run.sh --next 16               # real agent runs on the Next 16 variant
```

Results land in `evals/results/`. Compare them before and after changing any command, agent or skill. A change that lowers the pass rate or raises the cost is a regression.